

Lesson 9: Views, Procedural SQL and Database Design

Views

- In some cases you may want to store a query in the database
 - Our example from lab of supporters by country might be a use case
- Views can be used by applications, reporting/BI tools or even used to update data in tables
- It's typical for views to have some limitations in what they can do, but this will vary by DBMS

View Syntax

```
CREATE VIEW hfh.vw_supporter_by_country AS
  SELECT l.country_name
         ,COUNT(r.supporter_id) as count_supporters
  FROM hfh.country as l
  INNER JOIN
    hfh.supporter as r
  on l.country_code = r.country_code
  group by l.country_code;
```

Transaction Control (Flashback Lesson 7)

- Enables you to break commands into transactions to be committed or rolled back (undone).
 - Depending on the DBMS you can rollback the previous transaction or to a specific point in time.
- Transactions are typically the the safest way to INSERT, UPDATE and DELETE data.
- Data is not saved (or committed) to the database without a COMMIT statement

Transaction Control Example

```
#Being explicit and not using autocommit  
#Other languages use "BEGIN TRANSACTION"  
START TRANSACTION;
```

```
UPDATE database.table  
    SET column1 = 4  
        column2 = 'Updated all'  
        column3 = 'of the values'  
    WHERE column1 = 1;
```

```
COMMIT;
```

Procedural SQL

- Standard SQL doesn't enable us to close the gap with procedural languages using loops and conditional logic
 - SQL has been extended beyond "standard SQL" to enable us to get a bit closer to this
- Why does procedural SQL let us do?

Triggers

- A trigger is procedural SQL code that is invoked automatically by the DBMS when a DML event occurs
- They are used to enforce business rules and ensure data integrity
- Triggers are defined by:
 - i. which DML event triggers them
 - ii. when they occur relative to the DML event
 - iii. Whether they are implemented at the statement level or at the row level
 - iv. The action that is being taken
- Let's look at an example!

Triggers Continued

- Don't overuse triggers, but also don't write them off entirely
- There are many good arguments for using triggers or for implementing the same code in the application layer
- Syntax varies slightly by RDBMS, but the premise of what a trigger does remains the same

Trigger Syntax

```
/*This trigger insert the previous record values into an audit (or history table)*/  
CREATE OR REPLACE TRIGGER trg_ai_table_name_audit  
  AFTER INSERT ON db_name.table_name  
  FOR EACH ROW  
  INSERT INTO db_name.table_name_audit(id_column, column1, column2)  
    VALUES(NEW.id_column, OLD.column1, OLD.column2);
```

Stored Procedures

- Stored procedures are a named collection of SQL statements that are used to apply business logic
 - They can have both input and output defined
- They are stored on the RDBMS server
- They can be executed manually, scheduled or even added within a trigger
- The syntax varies by RDBMS, so we'll focus on MariaDB

Stored Procedures Continued

- Stored procedures can be used inside of triggers to ensure that
 - i. Code enforcing specific business logic needs to only be written once
 - ii. Multiple triggers based on different events can use the same code

Stored Procedure Syntax

```
DROP PROCEDURE IF EXISTS database.prc_example;

CREATE PROCEDURE database.sp_example(IN id_param INT
                                     ,OUT id_count INT)
    DELIMITER $$
    BEGIN
        SELECT count(*) as n
        FROM database.table
        WHERE id_column = id_param;
    END$$
    /*Reset the delimiter to ;*/
    DELIMITER ;
```

Executing a Stored Procedure

- In MariaDB we use the `CALL` command to execute a stored procedure
 - In other RDBMSs we might use `EXEC`

```
CALL database.prc_example(3, @id_count);
```

Stored/User-Defined Functions

- User defined functions are similar to stored procedures
- Where and how they can be used is dictated by the RDBMS
 - Unlike a stored procedures, you do not need a `CALL` command

Stored/User-Defined Functions Syntax

```
CREATE OR REPLACE FUNCTION database.fn_getage(birthdate DATE)
  RETURNS INT DETERMINISTIC
  DELIMITER $$
  BEGIN
    DECLARE @out_age INT;
    SET @out_age = ROUND(DATEDIFF(NOW(), birthdate)/365, 0);
    RETURN @out_age;
  END$$
  /*Reset the delimiter to ;*/
  DELIMITER ;
```

Static and Dynamic SQL

- **Static SQL** consists of the statements that we write and it doesn't change during execution
- **Dynamic SQL** can change during execution, for example we can build SQL statements on the fly depending on outputs and inputs
 - We won't cover this during the semester because it's more advanced
 - We need to be careful with Dynamic SQL because it can pose security risks if implemented poorly
 - Implementation varies across RDBMSs

Zooming out and Switching Gears

Information Systems (IS)

- A system that enables the collection, storage and retrieval of data
- Allows the transformation and management of data and information
- A database is just one piece of these information systems
- Typically an IS has:
 - A presentation layer like a website or app
 - An application layer which is where the majority of the logic and processes are implemented
 - The storage layer is our DBMS and enables us to capture and store data while ensuring data integrity

Getting to an IS

- **Systems Analysis** is the process by which requirements are gathered to determine the needs, architecture and extent of an IS (and whether it's needed)
 - Data modelling is a subset of this process
- **Systems Development** is the process that encompasses the implementation of the IS
 - Database Development is a subset of this process

Systems Development Life Cycle (SDLC)

- The SDLC is a collaborative and iterative cycle that contains multiple phases:
 - Planning
 - Analysis
 - Detailed Design
 - Implementation
 - Maintenance

SDLC Planning

- Focuses on the general overview of the organization and its needs
- It revolves around questions such as:
 - Is there an existing system filling this role?
 - If yes, should it be replaced or can it be modified?
 - Are there specific hardware/software requirements?
 - How many users are anticipated?
 - Will the IS be hosted on premises or cloud based
 - What are the costs?
 - How much is the hardware and software expected to cost (in \$) and how does this compare to the budget?
 - What are the costs of needed in terms of staff to implement and maintain the system?

SDLC Analysis

- Dives deeper into the end user needs and the short-comings of any existing IS
 - Starts to build our conceptual model (ERD) for the data
- This step asks questions such as:
 - What do the different groups of end users need to be able to do?
 - Does the current system address these, if not, what is not working?
 - Is the IS internal, external or both?
 - What are the current business processes and definitions that we hope to replicate in this IS?
 - How does this manifest itself in logic and in the overall design?

SDLC Design

- In this step the answers from the planning and analysis phases feed into the final design of the system which includes
 - The different components that make up the system
 - The logic and processes being built and how they map to the business processes
 - Screens and forms that will be visible to the end users and specific devices
 - The database design (detailed ERD)
 - Reporting needs
 - Training methods and materials

SDLC Implementation

- This step involves the iterative coding and testing
 - Adequate testing takes time and will include unit testing, data flow testing, end user testing and load/stress testing
- Once the database is implemented, the code will need to be written to load the data into the new database
 - This could be historic data
 - Or reference data like codes, countries, languages, etc.

SDLC Implementation Continued

- It's always best to build in logging and proactive alerting!
- During implementation it is common to:
 - Deploy between different non-production environments to test components and ensure a smooth deployment
 - Creating of a deployment run book which outlines the order of operations for deployment to production environments

SDLC Maintenance

- The need to perform maintenance will likely always exist and provides our entrance back into the SDLC
- Three main types of maintenance:
 - i. Corrective Maintenance is done in response to issues with the IS that are typically identified by Errors
 - ii. Adaptive Maintenance arise from changes in the business, which could result in changes to existing functionality or new functionality altogether
 - iii. Perfective Maintenance is done to enhance the existing system whether that's upgrading software, modify the hardware (on-prem or cloud) or making other minor changes to improve performance and/or functionality

Homework

- Read chapters 9 and 10